7. Gyakorlat

Python kurzus

JSON – FastAPI

JSON fájl írása

A JSON (JavaScript Object Notation) egy könnyen olvasható és feldolgozható adatcsere-formátum, amely a legtöbb webes alkalmazás alapvető adatformátuma.

A **json.dump()** függvénnyel az adatok szótárat JSON formátumban írjuk ki az adatok.json fájlba. Az **indent=4** a blokkok 4 karakteres behúzását eredményezi.

```
import json
adatok = {
   "nev": "Anna",
   "kor": 25,
   "varos": "Budapest",
   "hobbik": ["olvasas", "sport", "zenehallgatas"],
   "kedvenc_szamok": {
      "1": "Queen - Bohemian Rhapsody",
      "2": "Led Zeppelin - Stairway to Heaven",
      "3": "Pink Floyd - Comfortably Numb"
   }
}
# JSON fájl megnyitása írásra
with open("adatok.json", "w") as fajl:
   json.dump(adatok, fajl, indent=4)
```

adatok.json:

```
{
   "nev": "Anna",
   "kor": 25,
   "varos": "Budapest",
   "hobbik": [
      "olvasas",
      "sport",
      "zenehallgatas"
   ],
   "kedvenc_szamok": {
      "1": "Queen - Bohemian Rhapsody",
      "2": "Led Zeppelin - Stairway to Heaven",
      "3": "Pink Floyd - Comfortably Numb"
   }
}
```

JSON fájl olvasása

```
import json

# JSON fájl megnyitása olvasásra
with open("adatok.json", "r") as fajl:
    content = json.load(fajl)
    print(content)
    print(json.dumps(content, indent=4))
```

- A **json.load()** függvény a adatok.json fájl tartalmát beolvassa a **content** változóba Python **dictionary**-vé alakítva.
- A strukturált kiíráshoz a **json.dumps()** függvényt használjuk, az **indent=4** paraméterrel.

```
{'nev': 'Anna', 'kor': 25, 'varos': 'Budapest', 'hobbik': ['olvasas',
'sport', 'zenehallgatas'], 'kedvenc_szamok': {'1': 'Queen -
Bohemian  Rhapsody', '2': 'Led Zeppelin - Stairway to
Heaven', '3': 'Pink Floyd - Comfortably Numb'}}
{
    "nev": "Anna",
    "kor": 25,
    "varos": "Budapest",
    "hobbik": [
        "olvasas",
        "sport",
        "zenehallgatas"
    ],
    "kedvenc_szamok": {
        "1": "Queen - Bohemian Rhapsody",
        "2": "Led Zeppelin - Stairway to Heaven",
        "3": "Pink Floyd - Comfortably Numb"
    }
}
```

RESTful API bevezetés

1. RESTful API (Application Programming Interface)
A REST (Representational State Transfer) egy **architekturális stílus**, amelyet gyakran használnak web API-k tervezésére és fejlesztésére.
A REST-kompatibilis API **alapelvei:**

- **Kliens-szerver architektúra:** Az API külön választja a kliens és a szerver logikáját.
- **Állapotmentesség (statelessness):** Minden request független, a szerver nem tárolja az előző kérések állapotát.
- **Erőforrás alapú:** Az API erőforrásokat (resources) kezel, amelyek URL-ekkel azonosíthatók.
- **Egységes interfész (uniform interface):** Az API-n keresztül elérhető erőforrásokat egységes módon lehet kezelni.
- A műveletek általában a **HTTP módszerek**: **GET**: Adatok lekérése, **POST**: Új adat létrehozása, **PUT**: Adat frissítése, **DELETE**: Adat törlése.
- **JSON alapú kommunikáció**

2. HTTP protokoll (Hypertext Transfer Protocol) az API-k kommunikációs protokollja. Request – response modell: a kliens küld egy **kérés**t a szervernek, a szerver **válasz**t ad rá.

A HTTP kérés (request) részei:

- **URL:** Az erőforrás elérési útvonala
- **HTTP metódus:** A művelet típusa (pl. GET, POST,…)
- **Fejléc (headers):** Metaadatok a kérésről (pl. az adatok formátuma)
- **Törzs (body):** A kérés tartalma (pl. POST és PUT kéréseknél)

A HTTP válasz (response) részei:

- **Állapotkód (status code):** A válasz státusza (pl. 200 OK, 404 Not Found)
- **HTTP státuszkódok ismertetése:** https://www.w3schools.com/tags/ref_httpmessages.asp
- **Fejléc (headers):** Metaadatok a válaszról
- **Törzs (body):** A válasz tartalma, amely általában JSON formátumban van

3. FastAPI telepítése: pip install fastapi uvicorn

Egyszerű FastAPI alkalmazás létrehozása példa:

1. A FastAPI() példány létrehozza az alkalmazásunkat.
2. A **GET** metódus egy üdvözlő üzenetet ad vissza, ahol opcionálisan egy nevet is megadhatunk az URL-ben.
3. A **POST** metódus JSON adatokat vár a kéréstől, amelyet visszaküld a válaszban.

```
from fastapi import FastAPI
from typing import Optional
app = FastAPI()
@app.get("/udvozlet")
def udvozlet_nevvel(nev: Optional[str] = None):
    if nev:
        return {"uzenet": f"Üdvözöllek, {nev}!"}
    return {"uzenet": "Üdvözöllek, látogató!"}
@app.post("/adat")
def uj_adat_fogadasa(adat: dict):
    return {"uzenet": "Adat fogadva!", "kapott_adat": adat}
```

HTTP státuszkódok kezelése példa:

Ha a feltétel nem teljesül, akkor egy **404-es hibát** adunk vissza.

```
from fastapi import FastAPI, HTTPException
app = FastAPI()
@app.get("/adat/{adat_id}")
def adat_lekerese(adat_id: int):
    if adat_id == 1:
        return {"adat_id": adat_id, "nev": "Példa adat"}
    raise HTTPException(status_code=404, detail="Adat nem található")
```

4. Route-ok (útvonalak) kezelése FastAPI-ban, Pydantic modellek

A route-ok vagy útvonalak határozzák meg az alkalmazás különböző végpontjait (endpoints), minden útvonal **egy adott URL-hez és HTTP metódushoz kötött**.

```
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()

class Adat(BaseModel):                  # pydantic modell az adat validációjához
   nev: str
   eletkor: int

@app.post("/uj_felhasznalo")          # "/uj_felhasznalo": a végpont URL-je
def uj_felhasznalo(adat: Adat):
   return {"uzenet": f"Új felhasználó hozzáadva: {adat.nev}, életkor: {adat.eletkor}"}
```

A POST kéréssel új adatot várunk a klienstől JSON formátumban.

A **pydantic modell validálja** az adatot, biztosítja a megfelelő adatformátumot.

5. Response kezelés JSONResponse alkalmazásával

A response tartalmazhat **adatokat** (JSON formátumban), **HTTP státuszkódokat és fejléceket**.

Egyszerű válasz JSONResponse manuális beállítással:

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse
app = FastAPI()

@app.get("/json_valasz")
def json_valasz():
    tartalom = {"uzenet": "Ez egy JSON válasz!"}
    return JSONResponse(content=tartalom)
```

FastAPI alapértelmezésben automatikusan JSON válaszokat generál, de manuálisan is megadhatjuk a válasz formátumát: ebben a példában egy JSON választ adunk vissza manuálisan a **JSONResponse objektumként**.

HTTP státuszkód és fejléc beállítása a JSONResponse objektumban:

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse
app = FastAPI()

@app.get("/egyedi_valasz")
def egyedi_valasz():
   tartalom = {"uzenet": "Ez egy egyedi válasz!"}
   fejlecek = {"X-Special-Header": "Egyedi fejléc érték"}
   return JSONResponse(content=tartalom, headers=fejlecek, status_code=201)
```

A response-ban beállítunk egy content-et, egy egyedi fejlécet - headers (X-Special-Header) és egy egyedi HTTP státuszkódot (201). Ezzel **testreszabhatjuk** a válaszunkat.

Redirect válasz (átirányítás):

```
from fastapi import FastAPI
from fastapi.responses import RedirectResponse
app = FastAPI()
@app.get("/atiranyitas")
def atiranyitas():
   return RedirectResponse(url="/udvozlet")
```

A **RedirectResponse** által egy átirányítást hozunk létre, amely egy másik URL-re irányítja át a kérést. Most a **/udvozlet végpont** felé történik az **átirányítás.**

Példa a **response_model** paraméter alkalmazására:

FastAPI lehetőséget ad arra, hogy válaszainkat pydantic **modellekkel validáljuk,** ezzel meghatározzuk a visszaadott adatok formátumát.

```
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()

class ValaszModell(BaseModel):
    uzenet: str
    adat: dict

@app.get("/valasz_modell", response_model=ValaszModell)
def valasz_modell():
    return {"uzenet": "Sikeres válasz!", "adat": {"kulcs": "ertek"}}
```

A **response_model** paraméterrel megadtuk, hogy a visszaadott adat formátuma illeszkedjen a **pydantic modell**hez, biztosítva, hogy a válasz mindig a megfelelő szerkezetű legyen.

6. Egyszerű API létrehozása FastAPI-val példa

Az alkalmazás egy üdvözlő, és egy felhasználói adatokat visszaadó végponttal:

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List

app = FastAPI()                                # Alkalmazás inicializálása

class Felhasznalo(BaseModel):                  # Felhasználó modell
    nev: str
    eletkor: int

@app.get("/")                                  # Üdvözlő végpont
def udvozles():
    return {"uzenet": "Üdvözöllek a FastAPI alkalmazásban!"}

@app.get("/felhasznalok/", response_model=List[Felhasznalo])
def felhasznalok_listazasa():
    felhasznalok = [
        {"nev": "Anna", "eletkor": 25},
        {"nev": "János", "eletkor": 30}
    ]
    return felhasznalok
```

1. Létrehoztuk az alkalmazást egy FastAPI() példányként.
2. Létrehoztunk egy Felhasznalo nevű pydantic modellt.
3. Egy üdvözlő Root végpontot definiáltunk a GET metódussal, amely visszaad egy egyszerű üzenetet.
4. A /felhasznalok/ végpont visszaad egy listát a felhasználókról.

API futtatása lokálisan:

1. A szükséges csomagok telepítése: **pip install fastapi uvicorn**

(opcionálisan: pip install –r requirements.txt)

2. A FastAPI alkalmazást lokálisan futtatjuk az **uvicorn** szerver segítségével.
Az uvicorn egy gyors ASGI (Asynchronous Server Gateway Interface) szerver, amely képes futtatni FastAPI alkalmazásokat.

main.py:

```
from fastapi import FastAPI
from schemas.schema import ShopName
from routers.routes import routers as routes_router

app = FastAPI()
app.include_router(routes_router, tags=["Bolt"])

@app.get('/', tags=["Root"])
def route():
   return {'Welcome in ': ShopName}
```

- A projekt **main.py** nevű fájlja az API belépési pontja.
- Itt definiáljuk az alkalmazást.
- Hozzákapcsoljuk az útvonalakat: **app.include_router(…)**
- Definiálunk a Root-ban egy üdvözlést.

Parancs az API futtatásához: **`uvicorn main:app --reload --port 8000`**

- Az uvicorn main:app --reload parancs elindítja a FastAPI alkalmazást az uvicorn szerveren, ahol a **main** a fájl neve, és az **app** az alkalmazás objektum neve.
- A **--reload** kapcsolóval beállítjuk az automatikus újratöltést, amikor a fájlok módosulnak.

Az API elérhető lesz a http://127.0.0.1:8000 címen.

Az alkalmazás dokumentációja a FastAPI által automatikusan generált webes interaktív **Swagger UI** (User Interface) felületen érhető el.

Swagger UI elérése: **http://127.0.0.1:8000/docs**

Minta: models.py

```
from typing import List, Optional
from pydantic import BaseModel
class Oktato(BaseModel):
    nev: str
    email: str
class Hallgato(BaseModel):
    id: Optional[int]
    nev: str
    email: str
class Kurzus(BaseModel):
    id: Optional[int]
    nev: str
    tipus: str
    evfolyam: int
    nap_idopont: str
    helyszin: str
    oktato: Oktato
    hallgatok: Optional[List[Hallgato]]
    max_letszam: int
```

Minta: fajl_kezeles.py

```
import json
class KurzusFajlKezelo:
    ut = "kurzusok.json"

    def kurzusok_olvasas(self):
        try:
            with open(self.ut, "r") as be:
                kurzusok = json.load(be)
        except FileNotFoundError:
            return []
        return kurzusok

    def kurzusok_iras(self, kurzusok):
        with open(self.ut, "w") as ki:
            json.dump(kurzusok, ki, indent=4)
```

Példa: a model_dump() és a json.dumps() alkalmazása

model_dump(): A metódus egy Pydantic modell attribútumait egy **Python-szótárként** (dict) adja vissza. (Nem alakítja át az adatokat JSON formátumú szöveggé.)

```
@routers.post('/adduser', response_model=User)
def adduser(user: User) -> User:
    ###
    return JSONResponse(content=user.model_dump(), status_code=201)
```

json.dumps(): Python dictionary-t JSON-formátumú szöveggé alakít. Ha a user egy Pydantic modell, először a model_dump()-ot kell használni, hogy szótárat kapjunk, majd ezt átadhatjuk a json.dumps()-nak.

```
import json
from pydantic import BaseModel
class User(BaseModel):
    name: str
    age: int
user = User(name="Anna", age=25)
json_string = json.dumps(user.model_dump())
print(json_string)
```

```
{"name": "Anna", "age": 25}
```

Köszönöm a figyelmet!

Konzultáció

Minden héten csütörtökön 18:00 – 20:00